

The Zen of Programming

Table of Contents

1. Chapter 1: Getting Started

1.1 1. Installation & Environment

1.2 2. Your First Program

2. Chapter 2: Visual Data Flow

2.1 1. The Assignment Arrow (->)

2.2 2. The Return Arrow (<-)

3. Chapter 3: Types, The 5 Collections & Sentinels

3.1 Sentinels: Nothing vs Default

3.2 Type Casting (<: and Type(val))

4. Chapter 4: Control Flow, Ranges & Pattern Matching

4.1 Multi-Way Switch (check)

5. Chapter 5: Functions, Arrow Bodies & Closures

6. Chapter 6: The Three Container Tiers & Object Architecture

6.1 Enumerators (enumerator / enum)

7. Chapter 7: The Error Handling Triad

8. Chapter 8: Modules, Namespaces & Packaging

9. Chapter 9: Terminal Mastery & UI Canvas

10. Chapter 10: Dual Execution & The C Runtime

11. Chapter 11: Practical Projects

12. Chapter 12: Concurrency & The Unified Task Substrate

12.1 Structured Concurrency with with task_group

1. Chapter 1: Getting Started

Welcome to Zenlang! This chapter walks you through installing the compiler, writing your first program, and using the built-in developer tools.

1.1 1. Installation & Environment

```
# 1. Install the standalone compiler router
./scripts/zen install

# 2. Run your first script
./bin/zen examples/getting_started/01_hello.zl

# 3. Launch interactive REPL
./bin/zen repl
```

[bash]

1.2 2. Your First Program

```
use zen.io [zen]

function main() {
  io.writeln('Hello, Zenlang!')
  <- 0
}
```

2. Chapter 2: Visual Data Flow

Zenlang is founded on the principle of directional clarity: code syntax reflects the physical movement of data through variables, transformations, and function returns.

2.1 1. The Assignment Arrow (->)

Values flow left-to-right into their variable targets. Use `let` for mutable bindings and `set` for immutable constants:

```
let score -> 100 [zen]
score -> score + 25

set MAX_LIMIT : Integer -> 500
```

2.2 2. The Return Arrow (<-)

Functions emit results outward to their caller using the visual return arrow:

```
function multiply(a: Integer, b: Integer) : Integer { [zen]
  <- a * b
}
```

3. Chapter 3: Types, The 5 Collections & Sentinels

Zenlang provides a robust, optionally typed type system, capitalization-strict sentinels (Nothing and Default), and 5 distinct collection primitives.

Collection	Ordering	Size	Mutability	Literals
List	Ordered	Dynamic	Mutable	[1, 2, 3], List{...}, L{...}
Set	Unordered	Dynamic	Mutable	Set{1, 2}, S{1, 2}
Vector	Ordered	Fixed	Immutable	Vector{10, 20}, V{10, 20}
Tuple	Ordered	Fixed	Immutable	Tuple{1, 'a'}, (1, 'a')
Map	Unordered	Dynamic	Mutable	{ 'k' -> 'v' }, Map{...}

3.1 Sentinels: Nothing vs Default

Nothing denotes absence of a value (null/nil replacement). Default represents the type-declared zero state (0, 0.0, empty string, [], {}).

```
let avatar : String -> Nothing // Explicit absent state [zen]
let buffer : List -> Default // Materializes as []
let count : Integer -> Default // Materializes as 0
```

3.2 Type Casting (<: and Type(val))

```
let b : Byte -> 0x41 [zen]
let int_val -> b <: Integer // 65
let str_val -> b <: String // '0x41'
let num -> Integer('42') // 42
```

4. Chapter 4: Control Flow, Ranges & Pattern Matching

Clean conditional branching with when ... or, 4-boundary ranges, structured do loops with empty fallbacks, and multi-way check matching.

Range Operator	Name	Interval	Example
..	Range Between	(start, end)	1..5 -> [2, 3, 4]
..+	Inclusive End	(start, end]	1..+5 -> [2, 3, 4, 5]
..-	Exclusive End	[start, end)	1..-5 -> [1, 2, 3, 4]
...	Full Inclusive	[start, end]	1...5 -> [1, 2, 3, 4, 5]

4.1 Multi-Way Switch (check)

```
let code -> check cmd { [zen]
  case 'start' <- 0,
  case 'stop' <- 1,
  case 'restart' <- 2
} or <- -1
```

5. Chapter 5: Functions, Arrow Bodies & Closures

Functions feature parameterless declarations, return type positioning, single-expression arrow bodies, and snapshot-by-value closures.

```
// Parameterless function:
function say_hello { <- 'Hello' }

// Arrow function:
function square(x: Integer) : Integer -> x * x

// Arrow lambda:
let is_even -> function(x) -> (x % 2) == 0
```

[zen]

6. Chapter 6: The Three Container Tiers & Object Architecture

Zenlang unifies data structures into three distinct tiers:

Tier	Keyword	Memory Model	Primary Use
Low Tier	structure	Value semantics, flat C ABI	High-throughput records, Vector math
Middle Tier	object	Dynamic hash map, bound self	Fluid entities, 1:1 Zen Data (.zd) interop
High Tier	class	Nominal OOP, single inheritance	Class hierarchies, extends, init constructors

6.1 Enumerators (enumerator / enum)

```
enumerator Direction { North -> 1, South -> 2, East -> 3, West -> 4 }
let d -> Direction.North
io.writeln(d.name + ': ' + Str.to_string(d.value))
```

[zen]

7. Chapter 7: The Error Handling Triad

Zenlang rejects try/catch complexity, using an explicit error triad: raise (^), check (?), and assert (!).

```
function divide : [Decimal, Error] (a: Decimal, b: Decimal) {  
    when b == 0.0 { ^ 'DivisionByZero' with 'Zero divisor' }  
    <- a / b  
}  
  
// Graceful recovery:  
let safe -> check divide(10.0, 0.0) or 0.0  
  
// Invariant assertion:  
assert safe >= 0.0 raise 'NegativeResult'
```

[zen]

8. Chapter 8: Modules, Namespaces & Packaging

Modules organize code cleanly with use, from, and import. Every file has access to an intrinsic module metadata object.

```
use zen.io  
use zen.text.string as Str  
from zen.math.math import abs, min, max  
  
function main() {  
    io.info('Running module: ' + module.name)  
    io.info('Canonical path: ' + module.path)  
    <- 0  
}
```

[zen]

9. Chapter 9: Terminal Mastery & UI Canvas

Zenlang is a terminal-first programming language. Sub-millisecond ANSI canvas rendering, 60 FPS redraws, raw keyboard polling, and layout widgets are native standard library primitives.

```
use zen.ui.canvas as canvas  
use zen.sys.term as term  
  
function main() {  
    let cv -> canvas.create(80, 24)  
    canvas.draw_box(cv, 0, 0, 79, 23, 'rounded')  
    canvas.draw_text(cv, 10, 5, '* Zenlang Terminal Canvas *')  
    canvas.render(cv)  
    <- 0  
}
```

[zen]

10. Chapter 10: Dual Execution & The C Runtime

Every Zenlang program runs identically in the script VM during development and under ahead-of-time (AOT) C compilation for standalone binary distribution.

Pipeline	Command	Output	Target Use
Bytecode VM	zen script.zl	Direct execution	Rapid iteration, REPL
Native C AOT	zen compile main.zl -o app	Standalone ELF binary	Production services, CLI utilities

11. Chapter 11: Practical Projects

Real-world capstones combining file I/O, string parsing, color formatting, and terminal user interfaces into complete applications.

Projects include the zen-count Unix utility, log stream analyzers, and live interactive dashboards.

12. Chapter 12: Concurrency & The Unified Task Substrate

Zenlang eliminates the Function Color Problem (async/await) with a single task primitive running on lightweight M:N runtime fibers.

```
task fetch_profile : String (id: Integer) {                                     [zen]
  <- net.get('https://api.example.com/users/' + Str.to_string(id))
}

function main() {
  let h1 -> fetch_profile(101).spawn
  let h2 -> fetch_profile(102).spawn

  let p1 -> h1.wait
  let p2 -> h2.wait
  io.writeln('Profiles loaded cleanly')
  <- 0
}
```

12.1 Structured Concurrency with with task_group

Child tasks are bounded to their parent group lifetime. Any unhandled error cancels sibling tasks automatically with zero orphaned fibers.